

Modul 320

Objektorientiert Programmieren

Cyrill Gurtner (cygu)

8. Mai 2026

Lektionen

KW	Thema
02	Klasse vs. Objekt vs. Instanz, Attribute + Methoden
03	Methoden: Parameter, Rückgabewerte, Objektzustand
04	Kapselung: <code>private</code> , Getter/Setter, einfache Validierung
05	Konstruktoren: Default vs. eigener Konstruktor, Initialisierung von Objekten
08	Exceptions
09	Beziehungen: „hat-ein“ (Komposition, UML)
10	Repetition und Prüfungsvorbereitung
11	Vererbung
12	Prüfung 1: Klassen, Objekte, Methoden, Kapselung, Konstruktoren
13	Polymorphie, Overloading, Overriding
17	Einführung in Git und GitHub
18	Git Zusammenarbeit mit Branching, Pull-Requests
19	Abstrakte Klassen und Interfaces, Projektstart
21	Projektarbeit
22	Präsentationen

Abstrakte Klassen und Interfaces

Abstrakte Klassen und Interfaces

In dieser Lektion lernen wir:

- was eine abstrakte Klasse ist

- was eine abstrakte Methode ist

- warum abstrakte Klassen nicht direkt instanziiert werden können

- was ein Interface ist

- wie `extends` und `implements` verwendet werden

- wann man abstrakte Klassen und Interfaces verwendet

Kurze Repetition

Bisherige Begriffe:

Eine Klasse ist ein Bauplan

Ein Objekt ist eine konkrete Instanz einer Klasse

Attribute beschreiben Eigenschaften

Methoden beschreiben Verhalten

Eine Unterklasse kann mit `extends` von einer Oberklasse erben

Mit `@Override` kann eine Methode überschrieben werden

Beispiel:

Ein Hund ist ein Tier. Eine Katze ist auch ein Tier.

Problem: Zu allgemeine Klassen

Manchmal ist eine Oberklasse sinnvoll, aber als echtes Objekt zu allgemein.

Beispiele:

Tier

Fahrzeug

Medium

Zahlungsmittel

Frage:

Soll man wirklich ein allgemeines Tier erstellen können?

Abstrakte Klassen

Was ist eine abstrakte Klasse?

Eine abstrakte Klasse ist eine Klasse, die nicht direkt instanziiert werden kann.

Sie dient als gemeinsame Grundlage für Unterklassen.

Eine abstrakte Klasse kann enthalten:

- Attribute

- Konstruktoren

- normale Methoden

- abstrakte Methoden

Merke:

Eine abstrakte Klasse ist ein Bauplan für konkrete Unterklassen.

Syntax: abstract class

Eine abstrakte Klasse wird mit `abstract` markiert:

```
public abstract class Tier {  
    protected String name;  
  
    public Tier(String name) {  
        this.name = name;  
    }  
  
    public void anzeigen() {  
        System.out.println("Name: " + name);  
    }  
}
```

Nicht erlaubt

Bei einer abstrakten Klasse ist das nicht erlaubt:

```
Tier t = new Tier("Bello");
```

Das führt zu einem Fehler.

Warum?

Tier ist abstrakt und nur als gemeinsame Grundlage gedacht.

Konkrete Unterklasse

Eine konkrete Unterklasse kann von der abstrakten Klasse erben:

```
public class Hund extends Tier {  
  
    public Hund(String name) {  
        super(name);  
    }  
  
    public void bellen() {  
        System.out.println("Wuff!");  
    }  
}
```

Jetzt ist das erlaubt:

```
Hund h = new Hund("Bello");
```

Abstrakte Methoden

Was ist eine abstrakte Methode?

Eine abstrakte Methode hat keinen Methodenrumpf.

Sie sagt nur:

Diese Methode muss es geben

aber jede konkrete Unterklasse soll sie selbst umsetzen

Merke:

Eine abstrakte Methode ist eine Pflicht für Unterklassen.

Syntax: abstrakte Methode

Eine abstrakte Methode wird mit `abstract` markiert:

```
public abstract class Tier {  
    protected String name;  
  
    public Tier(String name) {  
        this.name = name;  
    }  
  
    public abstract void geraeuschMachen();  
}
```

Wichtig:

- keine geschweiften Klammern

- nur ein Semikolon

- die Klasse muss ebenfalls `abstract` sein

Methode implementieren

Die Unterklasse muss die abstrakte Methode überschreiben:

```
public class Hund extends Tier {  
  
    public Hund(String name) {  
        super(name);  
    }  
  
    @Override  
    public void geraeuschMachen() {  
        System.out.println("Wuff!");  
    }  
}
```

Ohne diese Methode wäre Hund selbst auch abstrakt.

Polymorphie mit abstrakter Klasse

Auch abstrakte Klassen können als Referenztyp verwendet werden:

```
Tier t1 = new Hund("Bello");  
Tier t2 = new Katze("Mimi");
```

```
t1.geraeuschMachen();  
t2.geraeuschMachen();
```

Die Variable hat den Typ `Tier`.

Das echte Objekt entscheidet aber, welche Methode ausgeführt wird.

Abstrakte Klassen zusammengefasst

Eine abstrakte Klasse:

- wird mit `abstract class` deklariert
- kann nicht direkt mit `new` erzeugt werden
- kann Attribute und Konstruktoren haben
- kann normale Methoden haben
- kann abstrakte Methoden haben
- wird mit `extends` erweitert

Merksatz:

Abstrakte Klasse = gemeinsame Grundlage mit teilweise offenem Verhalten.

Interfaces

Was ist ein Interface?

Ein Interface beschreibt, was eine Klasse können muss.

Es ist eine Art Vertrag.

Wenn eine Klasse ein Interface implementiert, verspricht sie:

- Ich habe diese Methoden

- Ich setze diese Methoden selber um

- Andere Klassen dürfen sich darauf verlassen

Merke:

Ein Interface beschreibt Fähigkeiten.

Beispiele für Interfaces

Interfaces passen gut für Fähigkeiten oder Rollen.

Beispiele:

Druckbar

Speicherbar

Bewertbar

Exportierbar

Suchbar

Idee:

Verschiedene Klassen können dieselbe Fähigkeit haben.

Syntax: Interface

Ein Interface wird mit `interface` deklariert:

```
public interface Druckbar {  
    void drucken();  
}
```

Das Interface sagt nur:

Jede Klasse, die `Druckbar` ist, muss `drucken()` haben.

Interface implementieren

Eine Klasse verwendet `implements`, um ein Interface umzusetzen:

```
public class Rechnung implements Druckbar {  
  
    @Override  
    public void drucken() {  
        System.out.println("Rechnung wird gedruckt.");  
    }  
}
```

`implements` bedeutet:

Diese Klasse erfüllt den Vertrag des Interfaces.

Polymorphie mit Interfaces

Ein Interface kann als Typ verwendet werden:

```
Druckbar d1 = new Rechnung();  
Druckbar d2 = new Bericht();
```

```
d1.drucken();  
d2.drucken();
```

Das funktioniert, weil beide Klassen Druckbar implementieren.

Abstrakte Klasse vs. Interface

	Abstrakte Klasse	Interface
Schlüsselwort	<code>abstract class</code>	<code>interface</code>
Verwendung	<code>extends</code>	<code>implements</code>
Direkt instanziiierbar	Nein	Nein
Attribute	Ja	normalerweise nein
Konstruktor	Ja	Nein
Normale Methoden	Ja	eher nein
Typische Idee	gemeinsame Grundlage	gemeinsame Fähigkeit

Praktikum